

Projeto do 4o Bimestre - Descrição das Entregas

1 - Visão Geral da Distribuição de Pontos

Entrega	Foco Principal	Pontuação
Entrega 1	Modelagem do Domínio e Encapsulamento	3,0 pontos
Entrega 2	Regras de Negócio e Tratamento de Erros Próprio	3,0 pontos
Entrega 3	Persistência Flexível (Node/SQLite ou Prisma)	4,0 pontos
Total		10,0 pontos

2 - Detalhamento das Entregas Semanais

Entrega 1: Modelagem de Entidades e Estrutura Inicial (3,0 Pontos)

O objetivo aqui é fazer o mapeamento do tema sorteado (ex: Organizador de Livro, Gestor de Finanças, etc.) para o paradigma orientado a objetos, preparando o terreno para a interface web.

- **Critério 1: Criação das Classes de Domínio (1,0 pt)**
 - Criação das classes principais do tema escolhido utilizando construtores (**Classes e objetos**). As propriedades devem cobrir o mínimo de 5 campos exigidos pelo formulário de cadastro (**estruturas de dados, objetos e tipos primitivos**).
- **Critério 2: Encapsulamento e Modificadores de Acesso (1,0 pt)**
 - Uso obrigatório de atributos privados (**#atributos**) e métodos de acesso (**getters e setters**) para proteger os dados sensíveis da aplicação (**Modificadores de acesso e atributos de classe**).
- **Critério 3: Modularização e Coleções (1,0 pt)**
 - Organização do projeto utilizando ES Modules (**import/export**) (**Módulos**). Utilização de arrays de objetos e manipulação por métodos nativos para simular a listagem antes do banco estar pronto (**Métodos de arrays e passagem por referência**).

Dica do Professor: O aluno já deve preparar a classe **Usuario** aqui, pensando nos campos necessários para a tela de login exigida no projeto.

Entrega 2: Lógica Avançada, Validações e Exceções (3,0 Pontos)

Nesta fase, as telas já começam a tomar forma. Nosso foco em POO será garantir que o back-end valide tudo com rigor científico (e sem travar a aplicação!).

- **Critério 1: Aplicação de Herança e Polimorfismo (1,0 pt)**
 - O projeto deve apresentar uma hierarquia de classes útil ao tema. *Exemplo:* Diferenciação entre **UsuarioAdmin** e **UsuarioComum** (essencial para definir o "Nível" na tabela de listagem), ou especializações dos itens do próprio tema (**Herança e Polimorfismo**).
- **Critério 2: Validação de Regras no Back-end e Exceções (1,0 pt)**
 - Implementação das regras de validação severas exigidas no back-end (formatos de e-mail, limites de valores, campos obrigatórios). Se a validação falhar, o sistema deve obrigatoriamente lançar um erro customizado usando **throw new Error()** (**Exceções e tratamento de erro**).
- **Critério 3: Resiliência no Fluxo da Aplicação (1,0 pt)**
 - Uso correto de blocos **try/catch** nos controladores/rotas para capturar esses erros e permitir que o sistema web reenvie a mensagem precisa de erro para a tela.

Entrega 3: Persistência de Dados e Fechamento (4,0 Pontos)

O objetivo desta etapa final é tirar os dados da memória RAM e garantir que o CRUD se comunique perfeitamente com o banco de dados escolhido pela equipe, suportando as rotas da interface web.

- **Critério 1: Configuração da Camada de Persistência (1,5 pts)**
 - **Opção Prisma ORM:** Configuração correta do arquivo `schema.prisma`, mapeando as entidades, tipos de dados e os relacionamentos de acordo com o tema escolhido.
 - **Opção Node/SQLite Nativo:** Criação de uma classe ou módulo utilitário de conexão (ex: `DatabaseConnection`) responsável por abrir a conexão com o arquivo SQLite e executar os scripts iniciais de criação das tabelas de forma estruturada.
- **Critério 2: Operações de CRUD Orientadas a Objetos & Rotas (1,5 pts)**
 - Os métodos de persistência devem estar encapsulados (em classes de serviço, DAOs ou Repositórios) para isolar as consultas SQL ou chamadas do Prisma da lógica das rotas.
 - A rota de exclusão `/excluirNomeCadastro` deve obrigatoriamente receber a chave primária vinda do front-end e executar a remoção com sucesso no banco de dados.
- **Critério 3: Gerenciamento de Estado e Qualidade do Código (1,0 pt)**
 - Implementação da lógica de verificação de credenciais no banco de dados para apoiar a criação e destruição das variáveis de cookies (status de logado) exigidas na regra de negócio do login.
 - Código limpo, legível, sem conexões abertas de forma redundante e com bom uso de métodos assíncronos (`async/await`).